

Pipeline de construction du graphe de co-vues

Enchaînement des fonctions, hypothèses de logique métier
et sources de variabilité entre deux exécutions

Neil Braun

Mirakl — Data Science / Target2Sell

Août 2026 — source : `graph_pipeline_prototype.ipynb`

Ce document part du seul notebook `graph_pipeline_prototype.ipynb`. Il isole d’abord les deux chaînes de fonctions qu’il contient — celle qui construit le **graphe** et celle qui construit les **arêtes d’entraînement** — puis retrace le notebook cellule par cellule en renvoyant, à chaque étape, aux hypothèses de logique métier numérotées [\[H1\]](#)...[\[H34\]](#) (sections 3 et 4). La section 5 recense tout ce qui peut être silencieusement exclu ([\[E1\]](#)...[\[E19\]](#)) et la section 6 explique pourquoi deux exécutions du notebook, même sur la même fenêtre de dates, ne donnent pas le même graphe ([\[V1\]](#)...[\[V14\]](#)).

Toute affirmation est tirée du code du notebook. Les rares points qui dépendent de l’environnement (version de Spark, rétention des tables) sont signalés comme *à vérifier* et non présentés comme relevés.

Table des matières

1	Les deux chaînes de fonctions	1
1.1	Chaîne A — construction du graphe	2
1.2	Chaîne B — construction des arêtes d’entraînement	3
1.3	Chaîne B’ — tables de référence production (split <i>val</i>)	4
2	Parcours du notebook, cellule par cellule	5
3	Hypothèses de la chaîne A — le graphe	8
4	Hypothèses de la chaîne B — les arêtes d’entraînement	11
5	Ce qui peut être exclu du graphe	13
6	Pourquoi deux runs donnent des graphes différents	15
6.1	Sources amont non figées	15
6.2	Amplification : une petite variation d’entrée, un grand écart en sortie	15
6.3	Non-déterminisme interne au notebook (mêmes entrées, sortie différente)	15
6.4	Paramètres qui changent le graphe à données constantes	16
6.5	Rendre un run reproductible	16
7	Les cinq points à trancher	16

1 Les deux chaînes de fonctions

Le notebook contient deux pipelines distincts qui ne se rejoignent qu’à la toute fin, dans l’objet `Data` de PyTorch Geometric :

Chaîne A — le graphe.

Tracking de navigation → sessions → paires de produits co-vus → `edge_index` d’entraînement
+ matrice de features `x`. C’est le graphe sur lequel circule le *message passing*.

Chaîne B — les arêtes d'entraînement.

Logs publicitaires (ads_adlog_fct) rapprochés du tracking → exécutions de recommandation → paires (trigger, candidat) étiquetées positif/négatif. C'est ce que la tête de prédiction apprend à classer, et ce sur quoi le MRR est calculé.

Ces deux chaînes n'ont pas la même sémantique d'arête : la chaîne A produit « ces deux produits ont été vus dans la même session », la chaîne B produit « ce produit a été recommandé à la suite de ce trigger, et vu / non vu » [H34].

1.1 Chaîne A — construction du graphe

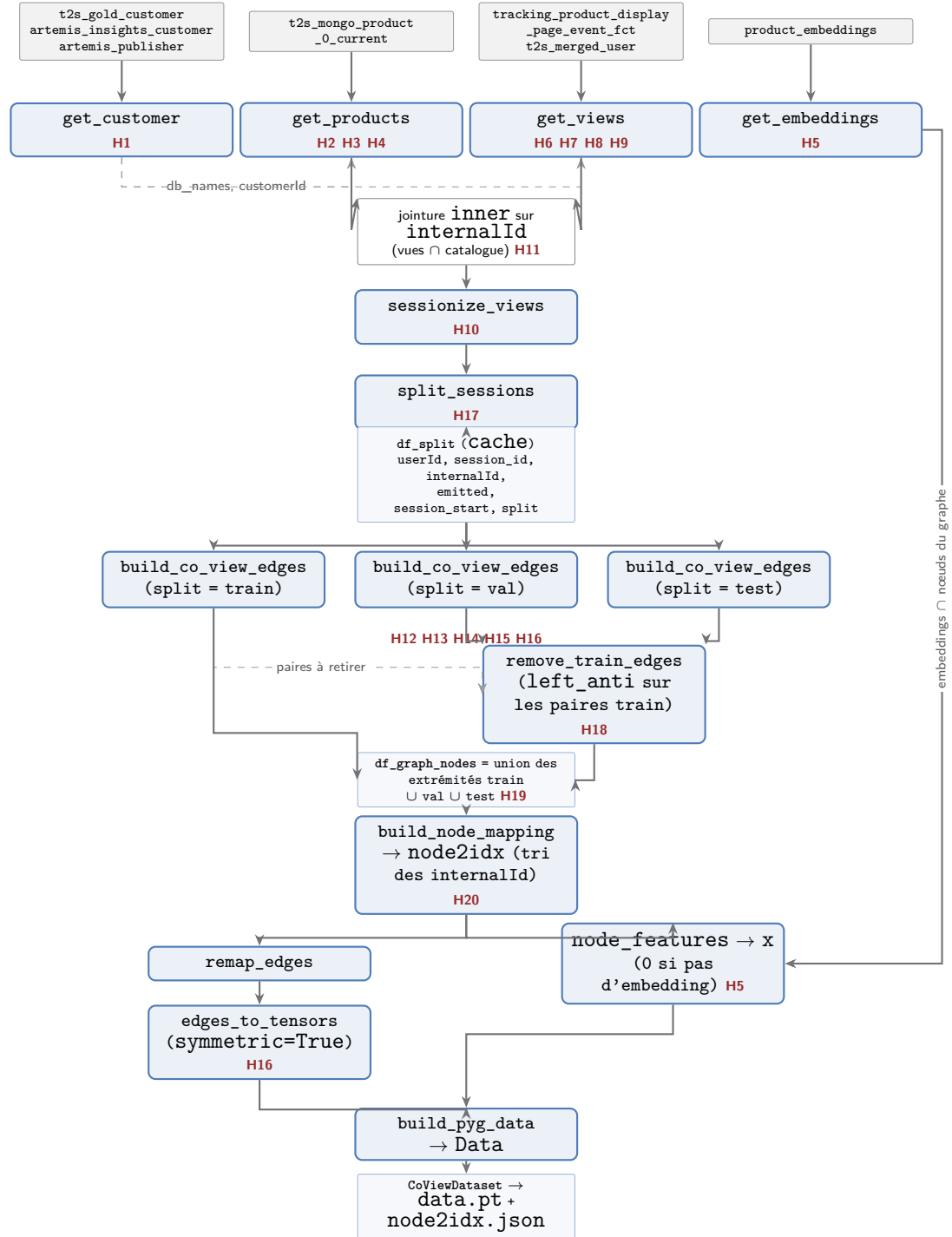


Figure 1 — Chaîne A. Les cadres bleus sont des fonctions du notebook, les cadres gris des tables Unity Catalog, les cadres clairs des artefacts intermédiaires. Les étiquettes rouges renvoient aux hypothèses du tableau 2. *df_val_edges* et *df_test_edges* ne servent qu'à élargir le jeu de nœuds et à produire les compteurs : ils ne sont pas passés à *build_pyg_data* (voir [H19]).

1.2 Chaîne B — construction des arêtes d'entraînement

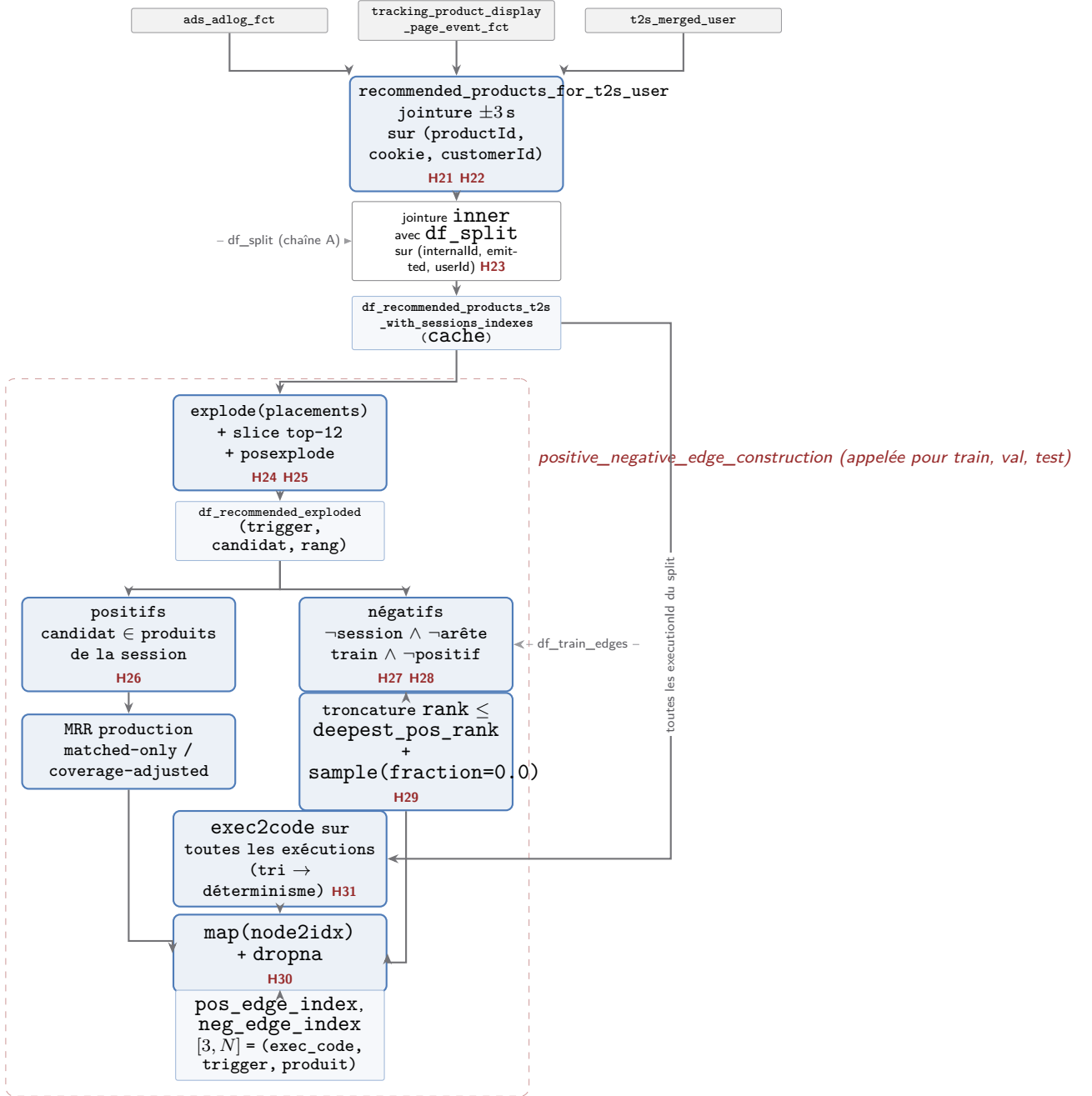


Figure 2 — Chaîne B. Le cadre pointillé délimite l'unique fonction *positive_negative_edge_construction*, appelée trois fois (train, val, test), toujours avec *df_train_edges* comme référence de négatifs.

1.3 Chaîne B' — tables de référence production (split *val*)

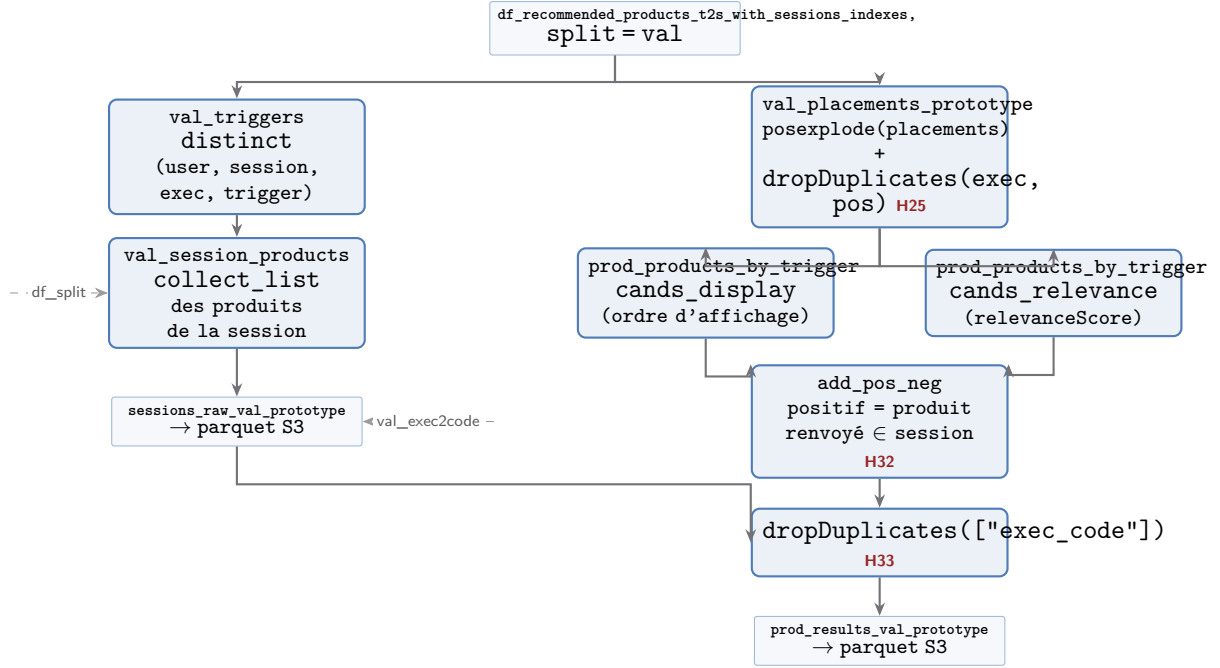


Figure 3 — Chaîne B'. Ces deux parquets ne servent pas à l'entraînement : ils donnent la baseline production sur *val*, en deux variantes d'ordre des 12 candidats. Aucun filtrage sur les arêtes du graphe n'y est appliqué, contrairement à la chaîne B ([H27] vs [H32]).

2 Parcours du notebook, cellule par cellule

Chaque étape est résumée en une ligne, suivie des hypothèses qu'elle pose. Les libellés complets sont dans les tableaux 2 et 4.

Cellules 0–2 — session Spark et définition des fonctions

Aucune exécution de logique métier : DatabricksSession, installation de `torch_geometric`, puis définition des sept fonctions de la chaîne A.

Cellule 3 — résolution du client et fenêtre temporelle

- `get_customer("maisons-du-monde")` puis `collect()` : les `customerId`, `publisherId` et `db_name` sont ramenés au driver sous forme de listes. [H1]
- `get_products` et `get_embeddings` sont câblés sur ces `db_names`. [H2] [H3] [H4] [H5]
- Fenêtre : `start_date = 2026-01-01`, `end_date = 2026-05-01`, donc `cutoff_train = 2026-04-01` et `cutoff_val = 2026-04-16`. Train $\approx$ 90 jours, val 15 jours, test 15 jours. [H9] [H17]
- `get_views` charge le tracking brut réconcilié par `t2s_merged_user`. [H6] [H7] [H8]

Cellule 4 — sessions, splits et arêtes de co-vue

- `df_views.join(df_products, "internalId", "inner")` : le filtre catalogue est appliqué *avant* la sessionisation, il peut donc scinder des sessions. [H11]
- `sessionize_views` : coupure à 1800s d'inactivité, fenêtre partitionnée par `userId` et ordonnée par (`emitted`, `internalId`). [H10]
- `split_sessions` : affectation train / val / test par `session_start`, puis `cache()`. [H17]
- `build_co_view_edges` sur chacun des trois sous-ensembles, puis `remove_train_edges` sur val et test. [H12] [H13] [H14] [H15] [H18]
- `count_nodes` sur l'union des trois jeux d'arêtes : le compteur affiché inclut donc des nœuds qui n'auront aucune arête d'entraînement. [H19]

Cellule 5 — passage en pandas

- `df_graph_nodes` = union des six colonnes d'extrémités (`train`, `val`, `test` $\times$ A, B), `distinct`. [H19]
- `toPandas()` sur les trois jeux d'arêtes : tout le graphe passe sur le driver. Les embeddings sont restreints par jointure `inner` aux nœuds du graphe.

Cellules 6–7 — indexation des nœuds

- `build_node_mapping` concatène les extrémités des trois jeux, `unique()` puis `sort()` : l'index est l'ordre croissant des `internalId`. [H20]
- `remap_edges` remplace les `internalId` par ces index. Un identifiant absent du mapping devient NaN — impossible ici, le mapping étant construit sur ces mêmes tables.

Cellule 8 — matrice de features

- `embedding_dim` est lu sur la *première* ligne de `emb_pandas` : dimension supposée homogène, garantie ici par le filtre sur `last_model_id`. [H5]
- `node_features` est initialisée à zéro puis remplie par affectation indexée. Un nœud sans embedding garde un vecteur nul, et le compteur `Nodes without embedding` le mesure. [H5]
- Si `emb_pandas` contenait deux lignes pour un même `internalId`, la dernière écrase la précédente, dans l'ordre de collecte de Spark — non déterministe. [V10]

Cellule 9 — tensorisation

- `edges_to_tensors(symmetric=True)` duplique chaque paire dans les deux sens : E double. [H16]
- `val_edge_index` et `test_edge_index` sont calculés ici mais ne sont **pas** passés à `build_pyg_data` (Cellule 25). [H19]

Cellules 10–11 — rapprochement avec les logs publicitaires

- `recommended_products_for_t2s_user` : jointure inner entre `ads_adlog_fct` et le tracking sur (`productId = fwProductId`, `userCookie = user.cookie`, `customerId`) dans une fenêtre de ± 3 secondes autour de `logInstant`, restreinte à `pageType = "PRODUCT"`. [H21] [H22]
- Jointure avec `df_split` sur le triplet exact (`internalId`, `emitted`, `userId`) : chaque exécution hérite de son `session_id` et de son `split`. [H23]

Cellule 12 — positifs, négatifs et MRR production

- Jointure `trigger × produits` de la session, sur (`userId`, `session_id`).
- `explode(sponsoredProductPlacementExecutions)`, puis `slice(relevantProducts, 1, 12)` et `posexplode` pour obtenir un rang 1-based. [H24] [H25]
- **Positif** : `internalId_session = internalId_candidate`, sans aucune contrainte d'ordre temporel entre la vue et la recommandation. [H26]
- MRR production : rang minimal d'un positif par `executionId`, en deux conventions (*matched-only*, comparable à `MRR_trigger` du modèle, et *coverage-adjusted*). Le dénominateur ne compte que les exécutions présentes dans `df_recommended_exploded`. [E14]
- **Négatif** : candidat non vu dans la session, non présent dans `df_train_edges` (testé dans les deux sens), et non déjà positif. Les positifs, eux, ne sont pas filtrés sur les arêtes existantes. [H27] [H28]
- Réduction du volume de négatifs : on ne garde que les rangs $\leq \text{deepest_pos_rank}$ de l'exécution, et les exécutions sans aucun positif sont échantillonnées à `fraction=0.0`, donc totalement écartées. [H29]
- `exec2code` est construit sur l'union positifs $\cup$ négatifs $\cup$ toutes les exécutions du split, puis *trié*
 - le commentaire du code indique explicitement que sans ce tri le code dépendrait de l'ordre de collecte de Spark. [H31]
- `map(node2idx)` puis `dropna` : les candidats absents du graphe disparaissent des positifs comme des négatifs. [H30]

Cellules 13–16 — sessions_raw_val_prototype

- `val_triggers` : triggers distincts du split val.
- `val_session_products` : pour chaque trigger, la liste (`internal_id`, `emitted`) de tous les produits de sa session — y compris ceux vus *après* le trigger. [H26]
- Jointure inner sur `exec_code` et contrôle de couverture (`n_triggers - n_sessions` doit valoir 0). Écriture en parquet sur S3.

Cellules 17–21 — prod_results_val_prototype

- `val_placements_prototype` : un placement identifié par (`executionId`, `placement_pos`), dédoublonné car la jointure session duplique les lignes adlog. [H25]
- Deux variantes du même tableau `relevantProducts` : `cands_display` (ordre du tableau) et `cands_relevance` (tri par `relevanceScore` décroissant, avec `internalId` comme départage). [H24]
- `prod_products_by_trigger` fusionne les placements d'un trigger, prend le rang minimal par produit, puis `row_number` et coupe à 12.
- `add_pos_neg` : positif = produit renvoyé appartenant à `session_ids`, sans aucun filtrage sur les arêtes du graphe. [H32]

- `dropDuplicates(["exec_code"])` : une exécution est réduite à un seul trigger, choisi arbitrairement. [H33]

Cellules 22–23 — diagnostic des deux variantes

Comparaison des ensembles `display / relevance` (`pct_identical_sets`, MRR dans les deux conventions). Si `pct_identical_sets` est proche de 1, les deux variantes sont équivalentes et la comparaison en aval est sans objet — c’est un test de l’hypothèse [H24].

Cellules 24–29 — assemblage et publication

- `build_pyg_data` place les co-vues `train` dans `edge_index` et les paires `ads` dans `train_pos_edge_index / train_neg_edge_index`. [H34]
- `CoViewDataset` sérialise `data.pt` et `node2idx.json`, puis `data_prototype.pt`, `node2idx_prototype.json` et `exec_mappings_prototype.json` sont poussés sur S3 sous des chemins suffixés `_prototype` — précisément parce que les `exec_code` diffèrent du pipeline principal. [H31] [V13]

3 Hypothèses de la chaîne A — le graphe

Id	Étape	Hypothèse métier	Si elle est fausse
H1	<code>get_customer</code>	Le <code>shortName</code> désigne un client unique, qui possède un <i>publisher</i> dans le namespace <code>artemis-prod</code> .	La jointure <code>inner</code> rend le client invisible, ou <code>collect()</code> renvoie plusieurs <code>db_name</code> et le graphe mélange deux catalogues.
H2	<code>get_products</code>	Le catalogue <i>courant</i> vaut pour toute la fenêtre historique : un produit actif aujourd'hui l'était déjà lors des vues de janvier.	Les produits désactivés depuis sont retirés du graphe alors que leurs vues existaient : le graphe rétrécit à mesure que l'on relance tard.
H3	<code>get_products</code>	Seuls les produits maîtres (<code>parentId = -1</code>) portent le signal ; le tracking a déjà ramené les vues de déclinaisons au maître.	Le signal des déclinaisons est perdu, ou au contraire agrégé alors que les déclinaisons se comportent différemment.
H4	<code>get_products</code>	<code>isActive</code> $\wedge$ <code>isOkInStream</code> = « produit recommandable » ; les autres n'ont pas à être des nœuds.	Des produits recommandés en production sont absents du graphe, donc non scorables (voir [H30]).
H5	<code>get_embeddings</code> , Cellule 8	Un seul modèle d'embedding (<code>last_model_id</code> figé), une ligne par produit, dimension homogène ; embedding absent $\rightarrow$ vecteur nul.	Doublons $\rightarrow$ écrasement non déterministe ; le vecteur nul est traité par le modèle comme un point du même espace, pas comme un « inconnu ».
H6	<code>get_views</code>	Un événement <code>product_display_page_event</code> = une vue produit. Pas de pondération par durée ou scroll, pas de déduplication des rafraîchissements.	Un rechargement de page compte comme une vue supplémentaire ; les vues de bots gonflent les sessions (voir [H14]).
H7	<code>get_views</code>	<code>t2s_merged_user</code> est fonctionnel (un <code>sourceId</code> $\rightarrow$ un <code>masterId</code>) et l'identité réconciliée est la bonne granularité de session.	Plusieurs <code>masterId</code> pour un <code>sourceId</code> dupliquent les lignes de vues et <i>doublent</i> les poids d'arêtes. Fusionner les appareils enchaîne des navigations parallèles dans une même session.
H8	<code>get_views</code>	Trafic non identifié (<code>user.internalId</code> null) et vue non résolue en produit maître ne portent pas de signal exploitable.	Une part importante du trafic — typiquement les visiteurs non logués — ne contribue à aucune arête.
H9	Cellule 3	<code>emitted.between(start_date, end_date)</code> : les bornes sont des <i>dates</i> converties en timestamps à minuit, donc la journée du <code>end_date</code> est de fait exclue.	La fenêtre réelle est de 120 jours et non 121 ; un décalage d'un jour entre le graphe et un autre calcul qui utiliserait la même variable.
H10	<code>sessionize_views</code>	Une session = 30 minutes d'inactivité, par utilisateur réconcilié, tous appareils confondus. Le tri (<code>emitted, internalId</code>) rend la coupure déterministe en cas d' <i>ex æquo</i> .	Un temps de réflexion long scinde une intention d'achat en deux sessions ; deux intentions rapprochées fusionnent. Voir aussi [V12] pour la sémantique du <code>cast("long")</code> .

Id	Étape	Hypothèse métier	Si elle est fausse
H11	Cellule 4	Le filtre catalogue s'applique <i>avant</i> la sessionisation.	Retirer une vue au milieu d'une session allonge l'écart entre ses voisines : une session peut se scinder en deux, ou perdre des produits. Le catalogue modifie donc la <i>sessionisation</i> , pas seulement le jeu de nœuds.
H12	build_co_view_edges	Deux produits vus dans la même session sont liés, indépendamment de l'ordre, de l'écart de temps et de la distance dans la session.	Une session de navigation exploratoire produit un graphe complet de produits sans rapport ; le signal séquentiel (A puis B) est perdu.
H13	build_co_view_edges	Une session compte une fois par paire (distinct avant la self-jointure) : le poids d'une arête = nombre de sessions distinctes contenant la paire.	Un utilisateur qui compare intensément deux produits ne pèse pas plus qu'une co-vue fortuite au sein d'une session.
H14	build_co_view_edges	USE_SESSION_WEIGHT_NORM = False : toutes les sessions pèsent pareil. Une session de n produits produit $\binom{n}{2}$ paires de poids 1.	Une session de 50 produits (bot, crawler, comparateur) apporte 1225 arêtes contre 1 pour une session de 2 produits : le graphe est dominé par les sessions longues. Dans cette configuration weight et raw_count sont identiques.
H15	build_co_view_edges	Avec un seuil minimal de poids : une co-occurrence unique est une arête à part entière.	Le graphe est majoritairement constitué d'arêtes de poids 1, c'est-à-dire de coïncidences — et ce sont exactement celles qui varient d'un run à l'autre ([V6]).
H16	build_co_view_edges edges_to_tensors	Une co-vue est symétrique et sans boucle : $A < B$ canonise la paire, puis symmetric=True la duplique dans les deux sens.	Aucune asymétrie de type « B est un accessoire de A » n'est représentable.
H17	split_sessions	Découpage temporel à la granularité <i>session</i> (session_start), pas à celle de l'événement.	Une session commencée avant cutoff_train et poursuivie après reste entièrement dans train : de l'information postérieure à la coupure entre dans l'entraînement.
H18	remove_train_edges	Une paire évaluée doit être inédite vis-à-vis du graphe d'entraînement.	Sans cela on mesurerait la mémorisation. Noter que val et test ne sont pas dédoublonnés <i>entre eux</i> : une même paire peut être dans les deux.

Id	Étape	Hypothèse métier	Si elle est fausse
H19	Cellule 5, build_node_mapping	Le jeu de nœuds est l'union train $\cup$ val test.	Des nœuds existent dans le graphe sans aucune arête d'entraînement (isolés), uniquement parce qu'ils ont été co-vus pendant val ou test. Ils augmentent <code>num_nodes</code> et donc la taille des couches d'embedding. Les tenseurs <code>val_edge_index</code> / <code>test_edge_index</code> de co-vue ne servent, eux, qu'aux compteurs.
H20	build_node_mapping	Les index de nœuds sont l'ordre croissant des <code>internalId</code> .	Reproductible à jeu de nœuds identique; mais l'ajout d'un seul produit décale tous les index suivants et rend un <code>data.pt</code> incompatible avec un <code>node2idx.json</code> d'un autre run ([V13]).

Tableau 2 — Hypothèses de la chaîne A.

4 Hypothèses de la chaîne B — les arêtes d’entraînement

Id	Étape	Hypothèse métier	Si elle est fausse
H21	<code>recommended_products</code>	Un <code>log</code> pour <code>2</code> <code>users</code> produit du même cookie, du même produit et du même client, séparés de moins de 3 secondes, sont le <i>même</i> événement.	Fenêtre trop large : plusieurs vues appartiennent à un même <code>log</code> et dupliquent les lignes. Trop étroite : un <code>log</code> est perdu par latence réseau ou décalage d’horloge. C’est le paramètre le plus sensible de la chaîne B.
H22	idem	Seules les pages <code>PRODUCT</code> portent une recommandation exploitable.	Les recommandations de la page d’accueil, de la recherche et des pages catégorie sont hors périmètre.
H23	Cellule 11	Le triplet exact (<code>internalId</code> , <code>emitted</code> , <code>userId</code>) identifie la vue déclencheuse, donc rattache l’exécution à sa session.	Un double envoi du tracking au même timestamp duplique l’exécution. Et si la vue déclencheuse n’a pas survécu au filtre catalogue ([H2]), l’exécution est perdue ([E11]).
H24	Cellule 12, Cellule 18	L’ordre du tableau <code>relevantProducts</code> est l’ordre d’affichage réel, et seuls les 12 premiers comptent.	Le commentaire du code prévoit explicitement la variante par score (<code>relevanceScore</code> , <code>performanceScore</code> , <code>fusionScore</code>); les cellules 17–23 la calculent et mesurent l’écart entre les deux ordres.
H25	Cellule 12, Cellule 18	Tous les placements sponsorisés d’une exécution sont fusionnés, le rang retenu est le rang minimal atteint. Le <code>dropDuplicates(["executionId", "placement_pos"])</code> suppose que les lignes dupliquées par la jointure session portent des <code>candidates</code> identiques.	Deux placements concurrents sur une même page sont traités comme une seule liste ordonnée.
H26	Cellule 12, Cellule 15	Positif = candidat recommandé <i>et</i> vu dans la même session — sans contrainte temporelle : un produit vu <i>avant</i> la recommandation compte comme positif.	C’est une fuite temporelle dans la définition du label : une partie des « clics prédits » sont des produits déjà visités, que le modèle peut retrouver par le graphe plutôt que par une vraie prédiction.
H27	Cellule 12	Négatif = candidat recommandé, non vu dans la session, <i>et</i> absent des arêtes d’entraînement (testées dans les deux sens).	Évite d’apprendre contre des faux négatifs, mais retire les négatifs les plus difficiles — ceux qui ressemblent le plus à des positifs.

Id	Étape	Hypothèse métier	Si elle est fausse
H28	Cellule 12	Asymétrie assumée : les négatifs sont filtrés sur les arêtes du graphe d'entraînement, les positifs ne le sont pas.	Les positifs peuvent être des arêtes déjà présentes dans <code>edge_index</code> , les négatifs jamais. Un modèle qui apprend seulement « cette paire est-elle une arête du graphe d'entrée ? » obtient un bon MRR sans généraliser. À quantifier : part des positifs déjà arêtes de train.
H29	Cellule 12	Négatifs tronqués au rang du positif le plus profond de l'exécution ; les exécutions sans positif sont écartées (<code>sample(fraction=0.0, seed=42)</code>).	Le modèle ne voit jamais de liste purement négative, alors que la production en produit. La <code>fraction</code> est un levier laissé à 0, pas une propriété du problème.
H30	Cellule 12	Un candidat absent de <code>node2idx</code> n'est pas scorable, donc <code>dropna</code> le supprime.	Le MRR du modèle porte sur une liste de candidats <i>filtrée</i> , le MRR production affiché juste au-dessus sur la liste <i>complète</i> : les deux ne sont pas strictement comparables tant que le taux de perte n'est pas mesuré.
H31	Cellule 12	<code>exec2code</code> couvre toutes les exécutions du split (indépendamment du label) et est rendu stable par tri.	Stable à jeu d'exécutions identique seulement : la moindre exécution en plus ou en moins renumérote tout et invalide les parquets déjà écrits ([V13]).
H32	Cellule 20	Pour les tables de référence production, le positif est défini par simple appartenance aux produits de la session, <i>sans</i> filtrage sur les arêtes du graphe.	Choix explicite dans le notebook : on veut le vrai retour production. Conséquence : la baseline et le modèle ne sont pas évalués sur exactement la même définition de positif ([H27]).
H33	Cellule 20	Une exécution = un trigger : <code>dropDuplicates(["exec_code"])</code> .	Si une exécution a été rattachée à plusieurs triggers ou à plusieurs sessions par la jointure $\pm 3s$, la ligne conservée est arbitraire, donc variable d'un run à l'autre ([V11]).
H34	Cellule 25	Le <i>message passing</i> circule sur les co-vues, la tête de prédiction apprend sur les paires ads : deux sémantiques d'arête coexistent dans le même objet <code>Data</code> .	Cohérent si les co-vues sont vues comme une structure de voisinage et les paires ads comme la tâche. Il faut simplement ne jamais interpréter un score comme « probabilité de co-vue ».

Tableau 4 — Hypothèses de la chaîne B.

5 Ce qui peut être exclu du graphe

Chaque filtre et chaque jointure `inner` du pipeline retire des lignes sans alerte. Le tableau ci-dessous les liste dans l'ordre du flux : c'est l'inventaire à instrumenter si l'on veut savoir combien de signal est perdu et où.

Id	Étape	Mécanisme	Ce qui disparaît
E1	<code>get_customer</code>	jointure <code>inner</code> clients / publishers	Un client sans <i>publisher</i> artemis-prod : le notebook s'exécute alors sur des listes vides sans erreur.
E2	<code>get_products</code>	<code>isActive</code> , <code>isOkInStream</code>	Produits désactivés ou hors flux au moment du run (pas au moment de la vue).
E3	<code>get_products</code>	<code>parentId = -1</code>	Toutes les déclinaisons.
E4	<code>get_embeddings</code>	<code>last_model_id</code> figé	Produits embeddés par une autre version : pas exclus du graphe, mais réduits à un vecteur nul.
E5	<code>get_views</code>	<code>user.internalId</code> non nul	Trafic non identifié.
E6	<code>get_views</code>	<code>masterProductInternalId</code> non nul	Vues non résolues en produit maître.
E7	<code>get_views</code>	<code>emitted.between(...)</code>	Journée du end_date (bornes prises à minuit, [H9]).
E8	Cellule 4	jointure <code>inner</code> vues / catalogue	Vues sur des produits absents du catalogue courant — et, par effet de bord, la sessionisation des vues restantes change ([H11]).
E9	<code>build_co_view_edges</code>	self-jointure <code>L.internalId < R.internalId</code>	Sessions à un seul produit distinct : elles ne produisent aucune arête, quel que soit leur volume.
E10	<code>remove_train_edges</code>	<code>left_anti</code>	Paires val/test déjà vues en train.
E11	<code>recommended_products</code>	<code>pageType = "PRODUCT"</code> , <code>fenêtre ±3s</code> , <code>publisherId</code> <code>isin(...)</code>	Recommandations hors page produit ; logs non appariés à une vue ; logs d'un <code>publisherId</code> non résolu.
E12	Cellule 11	jointure <code>inner</code> sur (<code>internalId</code> , <code>emitted</code> , <code>userId</code>)	Exécutions dont la vue déclencheuse n'a pas survécu à [E5] – [E8] .
E13	Cellule 12	<code>explode(sponsoredProductPlacements)</code>	Forcément Exécution(s) non sponsorisée(s) (tableau vide ou nul).
E14	Cellule 12	<code>slice(relevantProducts, 1, 12)</code>	Candidats au-delà du rang 12 — y compris les positifs qui s'y trouvaient, ce qui plafonne mécaniquement le MRR production.
E15	Cellule 12	<code>is_existing_edge</code>	Paires candidates déjà arêtes du graphe d'entraînement, côté négatifs uniquement ([H28]).
E16	Cellule 12	<code>rank ≤ deepest_pos_rank</code>	Négatifs plus profonds que le positif le plus profond.
E17	Cellule 12	<code>sample(fraction=0.0, seed=42)</code>	Tous les négatifs des exécutions sans positif.
E18	Cellule 12	<code>map(node2idx) + dropna</code>	Paires dont un produit n'est pas un nœud du graphe ([H30]).
E19	Cellule 20	<code>dropDuplicates(["exec_code"])</code>	Loggers surnuméraires d'une même exécution, choisis arbitrairement.

Tableau 6 — Points d'exclusion, dans l'ordre du flux.

Les deux exclusions à mesurer en priorité. [E8] conditionne tout l'amont : la taille du graphe est celle du catalogue au jour du run, pas au jour des vues. [E18] conditionne toute la comparaison au MRR production : tant que la proportion de candidats perdus n'est pas affichée, on ne sait pas si le modèle est comparé à la baseline sur la même population.

6 Pourquoi deux runs donnent des graphes différents

Le notebook ne fixe aucune version de table. Deux exécutions séparées de quelques jours, *sur la même fenêtre de dates*, lisent des données différentes. Les causes se rangent en quatre familles.

6.1 Sources amont non figées

- **[V1] Le catalogue est un instantané.** `t2s_mongo_product_0_current` est, comme son nom l'indique, l'état *courant*. `isActive`, `isOkInStream` et `parentId` évoluent : un produit désactivé entre deux runs disparaît du graphe, y compris pour les vues de janvier. C'est la première cause d'écart, et elle est monotone dans le temps — plus on relance tard, plus le graphe rétrécit.
- **[V2] La table d'embeddings s'enrichit.** Le filtre sur `last_model_id` fige le modèle, mais pas la population : les produits embeddés depuis le run précédent passent du vecteur nul à un vrai vecteur. Le compteur `Nodes without embedding` en est le témoin direct.
- **[V3] Le tracking reçoit des données tardives.** Événements arrivés en retard, retraitements et réécritures de partitions modifient le contenu d'une fenêtre historique déjà passée.
- **[V4] `t2s_merged_user` évolue.** La réconciliation d'identités s'améliore avec le temps : deux `userId` distincts hier deviennent un seul `masterId` aujourd'hui. Leurs vues fusionnent, les sessions changent, de nouvelles paires apparaissent — *sans qu'aucun événement nouveau n'ait été émis*. C'est la cause la plus contre-intuitive.
- **[V5] Rétention des logs publicitaires.** Si `ads_adlog_fct` a une durée de conservation bornée, relancer plus tard sur la même fenêtre donne moins d'exécutions, donc moins de positifs et un MRR production différent. *À vérifier* sur la table — le notebook n'en dit rien.

6.2 Amplification : une petite variation d'entrée, un grand écart en sortie

Le pipeline est quadratique par construction, ce qui transforme un écart marginal en entrée en écart massif en sortie.

- **[V6] Une vue de plus dans une session de n produits ajoute jusqu'à n arêtes.** Et comme aucun seuil de poids n'est appliqué (**[H15]**), ces arêtes de poids 1 entrent telles quelles dans le graphe.
- **[V7] Une fusion d'utilisateurs enchaîne deux sessions.** Si deux sessions de tailles a et b fusionnent, on passe de $\binom{a}{2} + \binom{b}{2}$ à $\binom{a+b}{2}$ paires, soit $a \times b$ arêtes créées d'un coup.
- **[V8] Retirer une vue intermédiaire peut scinder une session en deux (**[H11]**) :** les paires qui traversaient la coupure disparaissent toutes ensemble.
- **[V9] Les sessions longues dominant (**[H14]**) :** une seule session de bot en plus ou en moins déplace des milliers d'arêtes.

6.3 Non-déterminisme interne au notebook (mêmes entrées, sortie différente)

- **[V10] `node_features[positions] = emb_matrix` (Cellule 8) :** si un `internalId` a plusieurs lignes d'embedding, la dernière écrite gagne, dans l'ordre de collecte de Spark — qui n'est pas garanti.
- **[V11] `dropDuplicates(["exec_code"])` (Cellule 20) et `dropDuplicates(["executionId", "placement_pos"])` (Cellule 18) :** Spark conserve une ligne *arbitraire*, pas une ligne définie par un tri.
- **[V12] `(emitted - lag(emitted)).cast("long")` (`sessionize_views`) :** le résultat de la soustraction de deux timestamps, et sa conversion en `long`, dépendent de la version de Spark et du mode d'intervalles ANSI. Le code suppose des *secondes*, cohérent avec `session_timeout = 1800`. *À vérifier* avant tout changement d'environnement : une autre unité ferait de chaque vue une session isolée, et le graphe s'effondrerait silencieusement.

- **[V13] Couplage des artefacts.** `node2idx` ([H20]) et `exec_code` ([H31]) sont deux renumérations globales : elles sont stables à jeu constant, et entièrement décalées dès qu'un nœud ou une exécution apparaît. Un `data.pt` ne peut donc être lu qu'avec le `node2idx.json`, les `exec_mappings.json` et les parquets *du même run*. C'est exactement ce que dit le commentaire de la **Cellule 28** pour justifier le suffixe `_prototype`.
- **[V14] Un run long peut chevaucher une mise à jour amont.** `df_split` est mis en cache, mais `df_train_edges` ne l'est pas et il est réévalué à chaque usage (compteurs, `df_graph_nodes`, `toPandas`, puis trois appels à `positive_negative_edge_construction`). Si le cache est évincé ou si une table amont change en cours de run, deux parties du même graphe peuvent être calculées sur deux instantanés différents.

6.4 Paramètres qui changent le graphe à données constantes

Paramètre	Valeur	Effet
<code>session_timeout</code>	1800	Frontières de session, donc toutes les paires ([H10]).
<code>start_date / end_date</code>	2026-01-01 / 2026-05-01	Volume total ; dernier jour exclu ([H9]).
<code>cutoff_train / cutoff_val</code>	-30j / -15j	Répartition train/val/test ([H17]).
<code>USE_SESSION_WEIGHT_NORM</code>	False	Poids des sessions longues ([H14]).
<code>last_model_id</code>	06bbe57...	Population des nœuds avec features ([H5]).
fenêtre <code>adlog/tracking</code>	$\pm 3s$	Nombre d'exécutions appariées ([H21]).
<code>slice(..., 1, 12) / TOP_K</code>	12	Plafond du MRR production ([E14]).
<code>sample(fraction=...)</code>	0.0	Négatifs des exécutions sans positif ([H29]).
<code>symmetric</code>	True	Nombre d'arêtes doublé ([H16]).
seuil de poids minimal	<i>absent</i>	Bruit et instabilité du graphe ([H15]).

6.5 Rendre un run reproductible

Par ordre d'efficacité, et sans changer la logique métier :

1. **Figer les instantanés.** Lire chaque table en *time travel* Delta (`VERSION AS OF / TIMESTAMP AS OF`) et journaliser la version retenue à côté du `data.pt`. C'est ce qui neutralise [V1]–[V5] et [V14] d'un seul coup.
2. **Reconstruire le catalogue à la date de la vue** plutôt qu'au jour du run, si une table historisée existe — sinon assumer [H2] et le noter dans les métadonnées du dataset.
3. **Rendre les déduPLICATIONS déterministes** : remplacer les `dropDuplicates` de [V11] par un `row_number` sur un tri explicite, et dédoublonner les embeddings sur `max(last_update_date)` avant la **Cellule 8** ([V10]).
4. **Publier un manifeste de run** : versions de tables, valeurs des paramètres du tableau ci-dessus, `num_nodes`, nombre d'arêtes par split, `Nodes without embedding`, taux de perte de [E18]. Deux runs deviennent alors comparables même s'ils diffèrent.
5. **Instrumenter les exclusions** [E1]–[E19] par un compteur de lignes avant/après. Un écart inexpliqué entre deux runs se localise ainsi en une lecture.
6. **Introduire un seuil de poids minimal** ([H15]) et/ou une normalisation par taille de session ([H14]) : c'est le levier le plus direct pour stabiliser le graphe, puisqu'il supprime précisément les arêtes qui varient.

7 Les cinq points à trancher

Résumé pour décision, hors reproductibilité :

1. [H26] — un positif peut être un produit vu *avant* la recommandation. À restreindre à `emitted > logInstant` si l'on veut mesurer une prédiction et non une co-occurrence.

2. [H28] — les négatifs excluent les arêtes du graphe, les positifs non. Mesurer la part de positifs déjà présents dans `edge_index` : c'est le plafond de ce que le modèle peut obtenir sans généraliser.
3. [H30] et [E18] — MRR modèle sur candidats filtrés contre MRR production sur candidats complets. Afficher le taux de perte, sinon les deux chiffres ne se comparent pas.
4. [H14] et [H15] — absence de normalisation par session et de seuil de poids : le graphe est dominé par les sessions longues et les coïncidences.
5. [H19] — les nœuds val/test sans arête d'entraînement gonflent `num_nodes`. Décider s'ils doivent être des nœuds isolés du graphe d'entraînement ou en être retirés.